

UCS503P - Software Engineering

Vyapari

A Unified ERP Platform for Retail and Wholesale Businesses

Submitted by:

1024030169 Nandini Garg

1024030166 Vanshika

1024030163 Vanshika

Group: 3C16 BE Third Year, COE

Submitted to:

Miss Paramveer Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.1.1	Problem Statement	5
1.2	Project Objectives	6
1.3	Time-to-value	7
2	Methodology and Design	9
2.1	System Architecture	9
2.1.1	High-Level Design	9
2.1.2	Technical Stack	14
2.2	Implementation Details	15
2.2.1	Module 1: Core Functionality	15
2.2.2	Module 2: Secondary Features	16
3	Results and Evaluation	19
3.1	Testing Methodology	19
3.2	Performance Metrics	19
3.3	Analysis	20
4	Conclusion	21
4.1	Summary of Work	21
4.2	Key Findings	21
4.3	Future Work and Recommendations	21
4.4	Final Remarks	22

Introduction

1.1 Background and Context

Small and medium retail and wholesale businesses in India typically run their daily operations across several disconnected tools: a billing counter or register for sales, handwritten or spreadsheet stock records, a separate set of accounts for income and expenses, paper attendance and manual salary calculation for staff, and phone calls or chat messages for customer follow-up. Each of these records is maintained separately, so a single sale has to be reflected by hand in stock, in the accounts, and in the customer’s history. This is slow, error-prone, and gives the owner no single, current view of the business.

Vyapari is an integrated, web-based ERP (Enterprise Resource Planning) platform for such businesses. Its guiding idea is “one business, one system”: inventory, sales, invoicing, finance, customers, employees, payroll, analytics, and communication are built on one shared data model, so that an event in one module (for example, a completed sale) automatically updates every other module that depends on it (stock, invoice, ledger, customer history, dashboards, and notifications).

The system is designed around the workflow of an Indian retailer or wholesaler. It supports GST-aware invoicing with GSTIN and HSN/SAC details, Indian payment methods (Cash, UPI, Card, Bank Transfer, and Credit), amounts in Indian Rupees with Indian digit grouping, multiple outlets under one business, and four user roles (Owner, Manager, Cashier, and Accountant) with different levels of access. On top of the transactional core, Vyapari adds decision-support features: demand forecasting and reorder suggestions, RFM-based customer churn risk, cash-flow projection, a business-data assistant, and automated WhatsApp/SMS-style notifications.

The intelligence features use simple, explainable methods (moving-average forecasts, an RFM churn score, and a fixed-intent assistant) and are labelled as predictions or suggestions. Vyapari is a front-end prototype: a mock service layer with an in-memory store simulates the backend, WhatsApp/SMS delivery is simulated, and a synthetic dataset for a sample business, “Aarav General Store” (three outlets), is used for demonstration and evaluation.

1.1.1 Problem Statement

Small businesses today manage operations through scattered tools and manual effort. Specifically:

- Sales, stock, invoices, and accounts are recorded separately. A single sale must be re-entered or reconciled in several places, which takes time and leads to mismatches between stock levels, invoices, and ledger balances.

- Stock problems are noticed late. Owners find out about low stock or stock-outs after a customer has been turned away, and have no demand-based estimate of how much to reorder. Overstocking goes unnoticed in the same way.
- Outstanding payments and overdue invoices are tracked manually, which makes it hard to know how much money is owed and by whom.
- Customers who stop buying, or who stop buying a particular product, are not detected until the loss is visible in sales. There is no systematic way to identify at-risk customers or to follow up with them.
- Employee attendance and salary are handled manually, so salary deductions are difficult to trace back to attendance records.
- Customer and staff communication (stock alerts, order-ready messages, retention reminders) is manual and inconsistent, and does not respect whether a customer has agreed to receive messages.
- Owners lack a single, consistent view of sales, profit, receivables, and stock health, and different records often show different numbers for the same quantity.
- Staff with different responsibilities (cashier, manager, accountant) need different levels of access, for example a cashier should not see cost prices or salaries.

Vyapari addresses these problems by combining point-of-sale billing, inventory, orders, invoicing, customer management, finance, payroll, analytics, forecasting, an assistant, and notifications in one role-aware system with a single source of truth.

1.2 Project Objectives

1. **Objective 1: Unified transactional core.** Develop a POS workflow that automatically creates the sale, order, GST invoice, inventory update, and ledger entry, with a target median Time-to-Invoice (TTI) of ≤ 5 seconds.
2. **Objective 2: Multi-outlet inventory control.** Manage inventory across outlets with stock status tracking, movement history, transfers, CSV import/export, and barcode/QR scanning.
3. **Objective 3: Customer relationship and retention intelligence.** Maintain customer profiles with purchase history and lifetime value, and identify customer-product churn risk using an explainable RFM-based score.
4. **Objective 4: Finance and workforce management.** Provide finance, employee, attendance, and payroll management with attendance-based deductions and exportable GST reports.

5. **Objective 5: Decision support and communication.** Provide analytics, demand forecasting, reorder suggestions, cash-flow projection, a business assistant, and simulated WhatsApp/SMS notifications with customer opt-in, along with an in-app evaluation panel.

These objectives are SMART and map directly to Vyapari's modules, measurable system metrics, and the identified business problems.

1.3 Time-to-value

Development of Vyapari prioritises the core sale-to-invoice loop, because every other module reads from or writes to it. Work proceeds in the following order.

- **Foundation first.** The design system (colour, typography, spacing, and a reusable component library) and the data model with the mock service layer, event bus, and core business logic are built before any feature screen, so that all modules share one source of truth.
- **Access and shell.** Authentication, the six-step onboarding wizard, and the application shell with role-based navigation and global search are built next, so every later module is mounted inside a role-aware frame.
- **Core transactional workflow.** The Overview dashboard, Sales/POS, Inventory, Orders, and Invoices are built next. A single POS sale updates stock, creates the order and invoice, and posts to the ledger, which delivers the primary business value early. TTI is measured on this workflow.
- **Relationships and finance.** Customer management and churn intelligence, Finance, Employees and Attendance, and Payroll follow, consuming the data produced by the core workflow.
- **Intelligence and automation.** Analytics, Forecasting, the business-data assistant, and Notifications with simulated WhatsApp/SMS messaging are built on top of the accumulated data and events.
- **Scanning, reporting, and administration.** Barcode/QR scanning, Reports (including the GST summary), and Settings with adjustable role permissions complete the functional scope.
- **Data, integration, and quality.** The dataset is then expanded to full volume (50 customers, 150+ orders, and 90 days of history). A cross-module integration pass verifies that every number shown in more than one place agrees. Responsive and accessibility polish, final testing, and the evaluation panel close the work.

The integration pass is validated against an 18-step demo journey: log in as Owner, open the Overview, notice a low-stock item, open Inventory and the product detail, check the Forecast and reorder suggestion, open POS, scan or add the product, complete the sale, and then confirm that inventory, invoice, ledger, dashboard, customer purchase history, churn score, notifications, and Analytics all reflect the sale without a page refresh.

Methodology and Design

2.1 System Architecture

Vyapari follows a modular, layered, single-page web architecture. The layers are kept separate so that each can be developed and tested independently, and so that the mock service layer can later be replaced by a real backend without changing the user interface.

2.1.1 High-Level Design

At a high level, a React and TypeScript front end communicates with a service layer that exposes asynchronous functions modelled on a real API (for example, creating a sale, updating inventory, or listing products). Each call returns a Promise and simulates a network delay of 150–400 ms, so that loading states behave realistically. The service layer reads and writes a central in-memory store that is the single source of truth for all entities. Business rules (sale recording, churn scoring, forecasting, and payroll computation) are implemented once in this layer rather than inside screen components, so every module that shows the same quantity computes it through the same function.

The architecture has the following layers.

- **Presentation layer:** React pages for each module, mounted inside a shared application shell (sidebar, top bar, and global search), built on a common design system and UI component library.
- **Access-control layer:** a single permissions configuration maps every navigation item and sensitive page section to the roles allowed to see it. A route guard shows a “Not authorized” page when a user opens a link outside their role.
- **Service and business-logic layer:** mock API functions and core logic such as `recordSale()`, `computeChurnScore()`, `computeForecast()`, and the payroll computation.
- **Data layer:** the central in-memory store holding all entities, populated by a seed script that generates the synthetic dataset through the same service functions, so that the generated data is internally consistent. Data is held for the session and is not written to a database or to browser storage.
- **Event layer:** an internal event bus. Actions publish events (`sale.completed`, `stock.belowThreshold`, `order.statusChanged`, `invoice.overdue`, `churn.riskChanged`, `payroll.processed`), and the Notifications module subscribes to them.

- **Simulated external services:** a mock messaging service simulates WhatsApp/SMS delivery, and the browser camera is used for barcode/QR scanning, with a “Simulate scan” fallback when no camera is available.

The actors interacting with the system are the four user roles:

- **Owner:** has access to all modules, including finance, payroll (including approval), settings, user and role management, and the evaluation panel.
- **Manager:** works with sales, inventory, orders, customers, churn insights, forecasting, and analytics, and has a limited finance view without payroll amounts.
- **Cashier (Staff):** uses POS, orders, invoices, and customers, and has view-only access to inventory. Cost price and stock value are hidden, and finance and payroll are not accessible.
- **Accountant:** works with invoices, finance, expenses, reports, and payroll. POS is not accessible.

Simulated WhatsApp/SMS messages are produced by a mock messaging service and addressed to customers or to the Owner and Manager. Customers do not log in to the system.

Activity Diagram

Figure 2.1 traces the end-to-end sale workflow through Vyapari across four swimlanes (User, Frontend, Service Layer, and Event Bus with Notifications), from login through completing a POS sale to the stock update, invoice, ledger entry, and low-stock alert.

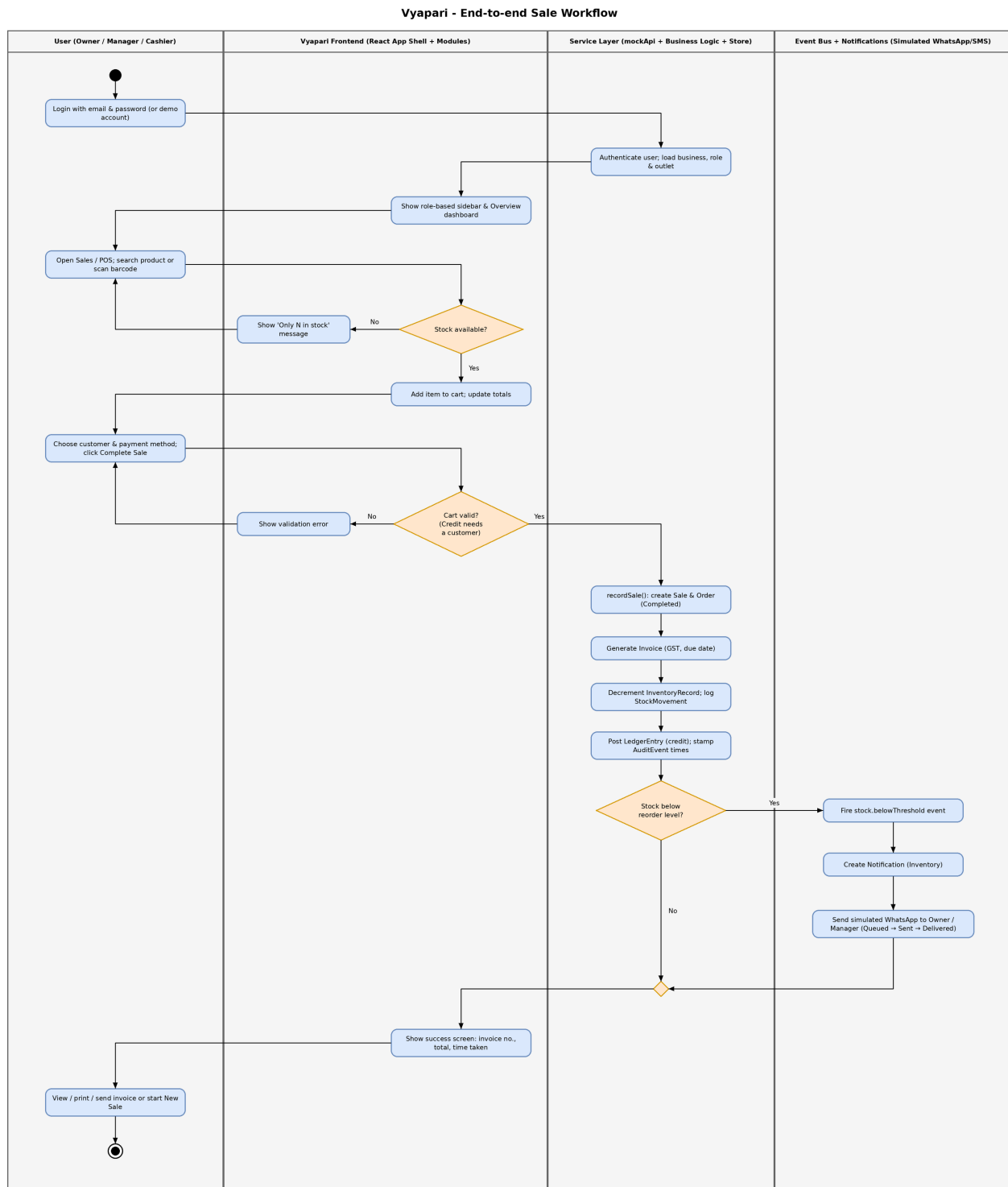


Figure 2.1: Vyapari activity diagram

Use Case Diagram

Figure 2.2 shows the use case diagram for Vyapari, covering the Cashier, Manager, Accountant, and Owner actors and the external actors the system depends on.

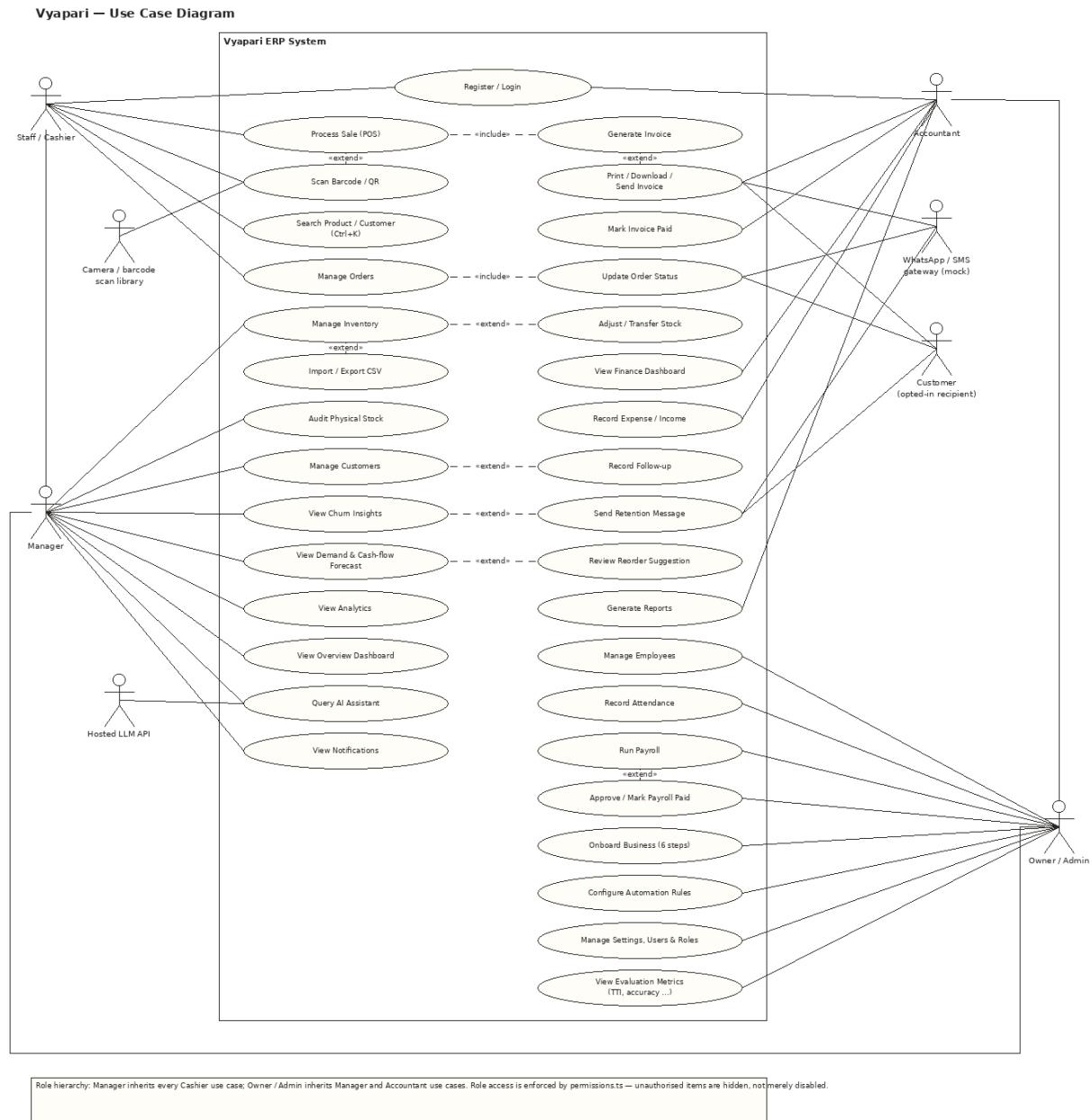


Figure 2.2: Vyapari use case diagram

Sequence Diagram

Figure 2.3 presents the sequence diagram for a typical session, covering login, a POS sale with an automatic low-stock alert, and the Owner checking the updated dashboard.

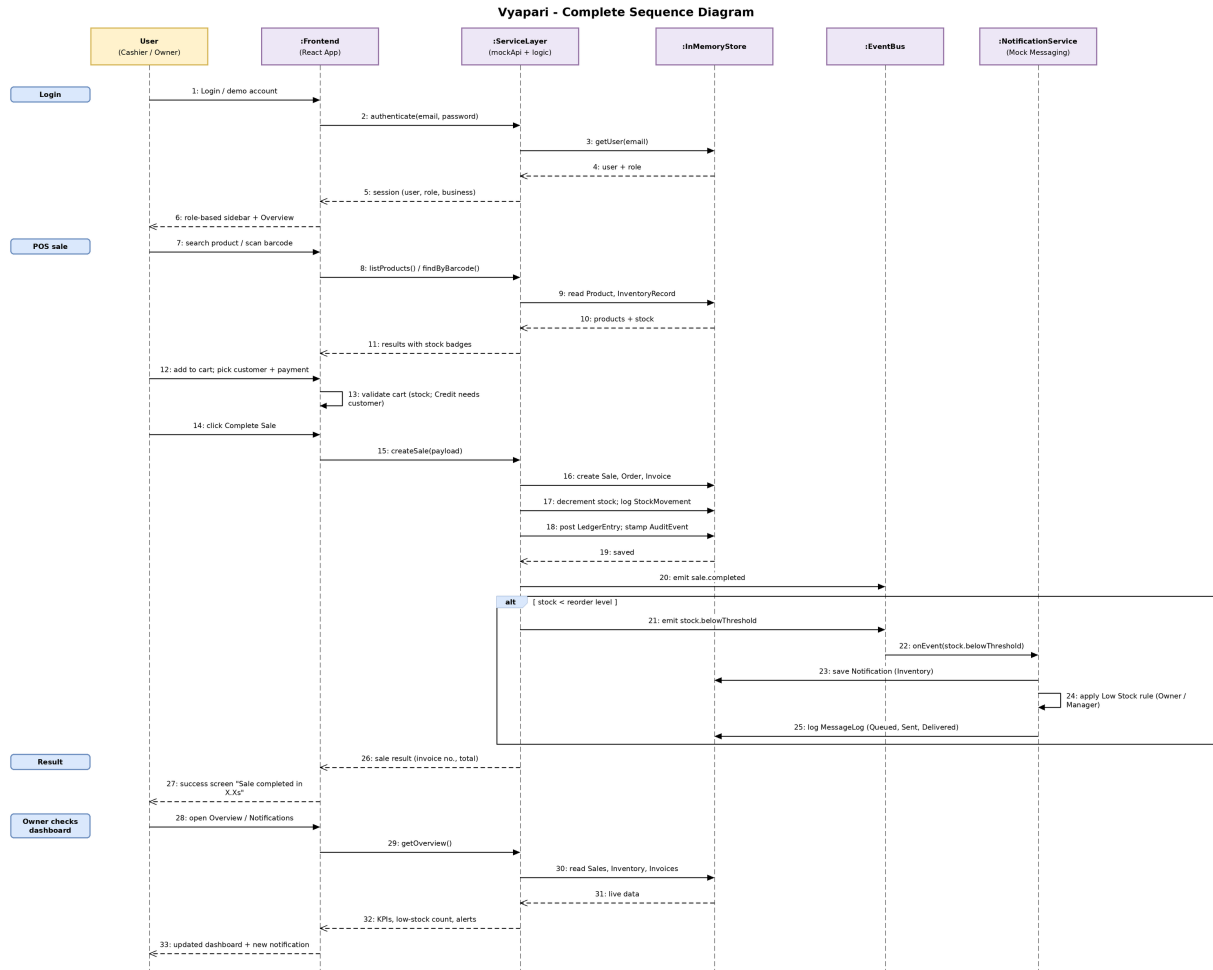


Figure 2.3: Vyapari sequence diagram

Class Diagram

Figure 2.4 shows the UML class diagram for Vyapari, a simplified view of the data model with 21 classes (including User, Business, Outlet, Product, InventoryRecord, Customer, Order, Sale, Invoice, LedgerEntry, Employee, PayrollRun, and Notification) along with their attributes, operations, and relationships.

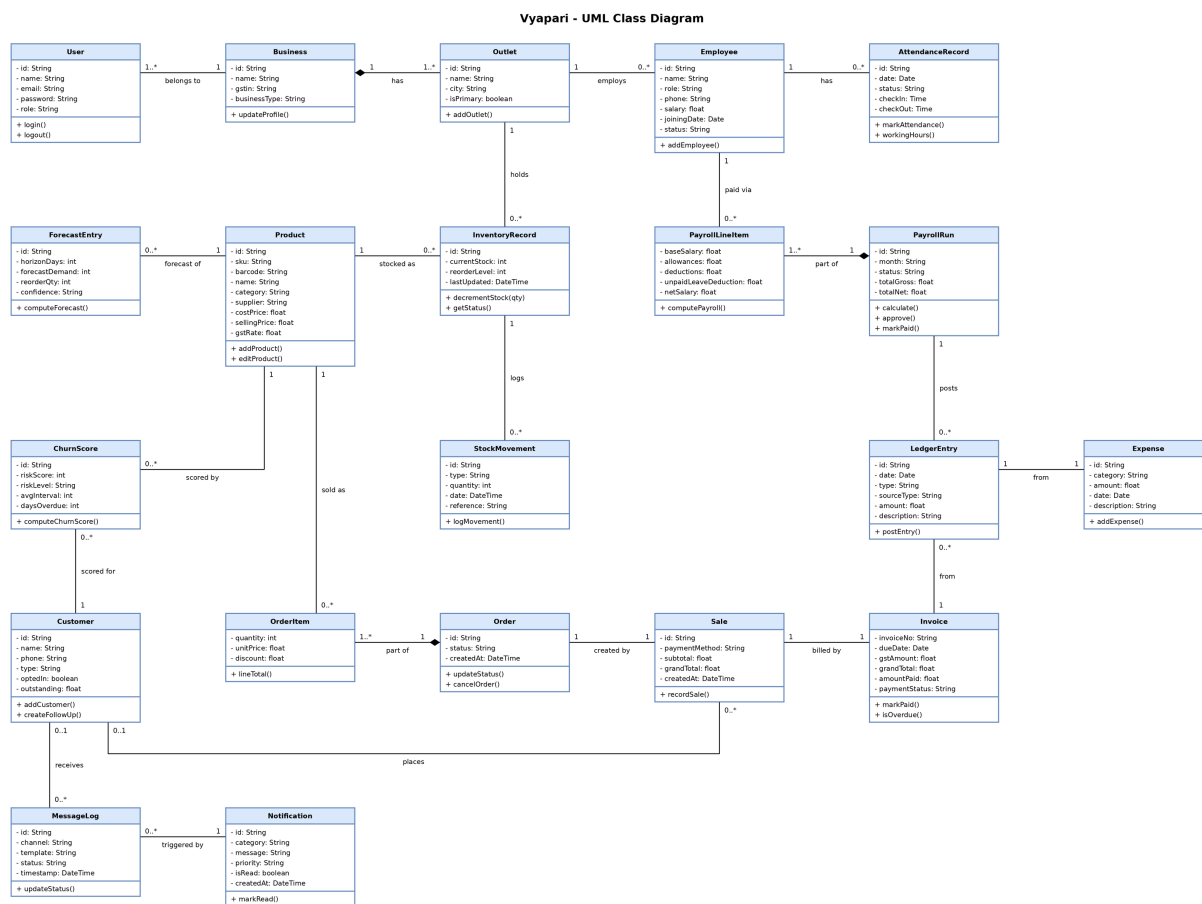


Figure 2.4: Vyapari UML class diagram

2.1.2 Technical Stack

- Framework/Language:** React with TypeScript and Vite. React Router v6 handles routing, while React Context with hooks manages state. ESLint, Prettier, and strict TypeScript support code quality.
- Data and Backend:** No database or server is used. A mock asynchronous service layer with simulated latency uses a central in-memory store and internal event bus.
- Intelligence:** TypeScript-based RFM churn scoring, moving-average/exponential-smoothing forecasting, cash-flow projection, and keyword/pattern-based assistant intent matching. No external ML or LLM service is used.
- Visualisation and UI:** Recharts for charts, Lucide for icons, and a custom design

system with reusable UI components.

- **Messaging:** Mock WhatsApp/SMS service that simulates delivery states; no real provider is connected.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core of Vyapari connects Sales/POS, Inventory, Orders, and Invoices, with results reflected on the Overview dashboard.

Sales / Point of Sale. The POS supports product search by name, SKU, or barcode, cart management, discounts, customer selection, and Cash, UPI, Card, Bank Transfer, and Credit payments. GST and total amounts are calculated automatically. Stock shortages are blocked and duplicate sales are prevented during processing. When a sale completes, the following chain runs:

Cart $\rightarrow$ `recordSale()` $\rightarrow$ Sale + Order (Completed) + Invoice $\rightarrow$ Inventory Decrement $\rightarrow$ Ledger Entry $\rightarrow$ Audit Timestamps $\rightarrow$ Events Published

The success screen displays Time-to-Invoice and invoice actions such as view, print, download, and simulated sending or starting a new sale.

Inventory. Inventory is maintained per product and outlet with SKU, barcode, category, supplier, pricing, GST rate, reorder level, and stock information. Stock is classified as Out of Stock, Critical, Low Stock, Healthy, or Overstock. Stock movements, outlet transfers, and insufficient-stock checks are supported. Products with sales history are archived instead of deleted, and Cashiers cannot view cost price or stock value.

Orders. Orders progress through Pending, Confirmed, Processing, Ready, and Completed, with cancellation and return support. POS sales automatically create completed orders, while manual orders can also be created. Timelines record status changes, cancellations/returns update inventory and ledger entries, and Ready orders generate notifications.

Invoices. Each sale generates an Indian GST invoice containing business details, GSTIN, customer details, HSN/SAC, items, discounts, GST, payment details, and balance due. Invoices support Paid, Pending, Overdue, and Partially Paid statuses, printing, and payment updates that affect the ledger and customer balance.

Overview Dashboard. The dashboard displays sales, orders, gross profit, low-stock items, outstanding payments, sales trends, inventory health, action items, and recent activity. Values are derived from the central store to maintain consistency across modules.

2.2.2 Module 2: Secondary Features

Beyond the core loop, several supporting modules complete the platform:

- **Authentication, Onboarding, and Roles:** Provides demo login, six-step onboarding with validation, role-based navigation for Owner, Manager, Cashier, and Accountant, unauthorized-page protection, and global search using **Ctrl+K**.
- **Customer Management (CRM):** Maintains Retail/Wholesale customers with purchases, outstanding balance, status, churn risk, lifetime value, orders, average order value, purchase history, preferred products, follow-ups, and message opt-in.
- **Customer Churn Intelligence:** Calculates customer-product churn risk using purchase history and repurchase intervals. Scores range from 0–100 and are classified as High, Medium, or Low, with explanations and revenue-at-risk information. Insufficient history is reported as “Not enough history yet”, and results are labelled as Predictions or Suggestions.
- **Finance:** Provides revenue, expenses, profit, receivables, payables, cash position, financial charts, ledger management, filtering, and CSV export. Sales, invoice payments, and payroll can automatically update the ledger.
- **Employees, Attendance, and Payroll:** Manages employee records, attendance, working hours, salary calculations, deductions, and unpaid-leave adjustments. Payroll follows Draft, Calculated, Approved, and Paid, with approval/payment restricted to Owner and Accountant. Payslips can be printed.
- **Analytics and Reports:** Provides Sales, Product, Customer, and Financial analytics, including best/slow-moving products, inventory turnover, revenue contribution, customer comparisons, and period comparisons. Reports include Sales, Inventory, Profit and Loss, Customer, Outstanding Payments, Attendance, Payroll, and Tax/GST reports with CSV export and printing.
- **Forecasting:** Forecasts product demand for the next 14 days using moving average or exponential smoothing and provides reorder suggestions with confidence levels. Users can accept or reject suggestions. Cash-flow forecasts cover 7, 30, or 90 days, and forecast-versus-actual performance is tracked.
- **Business Assistant:** Provides fixed-intent answers for sales summaries, low stock, reorder suggestions, order status, at-risk customers, expenses, unpaid invoices, and

FAQs. Responses use structured cards/tables, while unmatched queries are logged as unresolved.

- **Notifications and Messaging Automation:** Provides notifications for Inventory, Orders, Customers, Finance, and System events. Automation rules include Low Stock, Order Ready, and Customer Retention. Messages progress from Queued to Sent and Delivered/Failed, with retry and duplicate prevention. Messages are skipped when customers have not opted in.
- **Barcode/QR Scanning:** Provides barcode/QR scanning in POS and Inventory. Valid scans add/open products, invalid scans allow manual search, and repeated scans increase quantity. Scan-based and manual billing times are recorded.
- **Settings:** Includes business profile, outlets, users and roles, GST/tax defaults, invoices, notifications, barcode settings, security, and integrations. WhatsApp, SMS, and payment gateways are simulated. Role changes update access, GST changes do not affect previous invoices, and the last Owner cannot be removed.
- **Synthetic Dataset and Evaluation Panel:** Uses a consistent synthetic dataset containing a business with three outlets, users, employees, products, customers, orders, sales, invoices, expenses, attendance, and payroll data. Edge cases include different stock states, overdue/partial invoices, opted-out customers, and invalid barcodes. The Owner-only evaluation panel measures Time-to-Invoice, stock accuracy, forecast acceptance, churn detection, assistant resolution, notification delivery, and scan-to-bill time.
- **Responsive Design and Accessibility:** Supports desktop, laptop, tablet, and mobile layouts with responsive navigation. Accessibility includes keyboard navigation, visible focus indicators, semantic HTML, labelled controls, and WCAG AA colour-contrast checks.

Results and Evaluation

3.1 Testing Methodology

Testing so far has been manual and was done after each implementation step.

- **Unit Testing:** Not yet done. Each module was checked against the acceptance criteria written for its step, for example that a sale reduces stock and that a risk score matches its listed factors.
- **Integration Testing:** Checked by hand after each step. We completed a sale in the POS and confirmed that the order, invoice, stock level and ledger entry were all updated. We also confirmed that the same figures appear consistently in the Overview, Inventory and Finance pages.
- **Performance Testing:** The POS success screen shows the elapsed time for each sale, and timestamps are recorded for time-to-invoice. A formal measurement run has not been done yet.
- **User Testing:** Tested user workflows across different roles (Owner, Manager, Cashier, Accountant) to verify permissions and ease of use.

3.2 Performance Metrics

Table 3.1 lists the evaluation metrics from the project proposal and their current status. We have not recorded measured values yet, so none are reported here.

Metric	Target	Status
Time-to-invoice	median ≤ 5 s	Timestamps logged; to be measured
Forecast usefulness	Suggestions accepted	Accept/reject logged
Chatbot resolution	Answered without escalation	Queries logged
Notification reliability	Messages delivered	Delivery states logged
Stock accuracy	Match with physical count	Pending
Scan-to-bill speed	Faster than manual entry	Pending

Table 3.1: Evaluation Metrics and Current Status

3.3 Analysis

- **Objectives:** The core modules and the main sales-to-finance flow are built and working together. Barcode scanning, reports, settings and the final evaluation of the metrics above are still to be done.
- **Challenges:** Keeping many modules consistent with each other, so that a change in one place (such as a sale) is reflected correctly everywhere else.
- **Resolution:** All business logic and shared data are kept in one service layer, and each feature has its own module folder. This keeps the modules independent while the numbers stay consistent across pages.
- **Next step:** Full testing and measurement of the metrics will follow the integration, polish and testing steps.

Conclusion

4.1 Summary of Work

The objective of this project was to build Vyapari as one integrated web-based ERP prototype for retail and wholesale businesses, replacing separate tools for billing, stock, accounts, customers and staff. Up to this stage, the core modules are in place: POS, Inventory, Orders, Invoices, CRM and churn insights, Finance, Employees and Payroll, Analytics, Forecasting, the AI Assistant, and Notifications. Together they support the main chain from Inventory to Sales, Invoice, Finance and Analytics. Barcode scanning, Reports, Settings, full demo data, integration checks and final testing are still pending.

4.2 Key Findings

- **Finding 1:** A single sale updates the whole system. Completing a sale in the POS creates the order and invoice, reduces stock, posts a ledger entry and records timestamps, so the other modules stay consistent and time-to-invoice can be measured.
- **Finding 2:** Keeping all business logic in one service layer, with each feature in its own module folder, lets modules share data and events without being tangled together. This keeps later UI changes isolated to one module.
- **Finding 3:** Predictive features are safe to use when presented as suggestions. Churn scores use transparent RFM-based logic with a “Why?” explanation, forecasts use simple moving-average methods, no purchase order is placed automatically, and messages are sent only to opted-in customers.

4.3 Future Work and Recommendations

1. Add camera-based barcode/QR scanning to the POS and Inventory, with manual fallback and scan-to-bill timing [Tea24]
2. Build the Reports centre (including the GST summary) and the Settings and role-permission screens
3. Expand the demo data to the full planned volume so every status, edge case and chart is exercised

4. Run the cross-module integration pass to confirm that counts and totals match everywhere they appear
5. Complete responsive and accessibility polish, then run the full manual testing checklist
6. Add the evaluation panel to report time-to-invoice, stock accuracy, forecast acceptance, chatbot resolution and notification delivery
7. Replace the mock messaging, assistant and data services with real integrations once they are available

4.4 Final Remarks

This work showed that a unified ERP is mainly an integration problem: individual screens matter less than how each action flows into the other modules. The prototype has reached the point where a realistic business journey can be followed across most modules. The remaining work is about completing, verifying and measuring that integration rather than adding new ideas.